MagicCode Super-Resolution Software Integration Guide

Document Version: v3.0.0. Release Date: October 24, 2026.

1. Libraries

Platform	Archive	Notes
Android	lib/android/libmagic_sr.a	NEON CPU; Vulkan / OpenGLES GPU; graphic GPU yes; FG not in this Android Vulkan build
iOS	lib/ios/libmagic_sr.a	NEON CPU; Metal GPU; graphic GPU yes; FG not available
macOS arm	lib/mac_arm/libmagic_sr.a	NEON CPU; Metal GPU; graphic GPU yes; FG not available
Windows	lib/windows/libmagic_sr.dll, .lib, .dll.lib	x86 CPU; Vulkan / Direct3D 11 / OpenGL 4.3 GPU; graphic GPU yes; FG on Vulkan
Linux x86_64	lib/linux/libmagic_sr.a / .so	x86_64 CPU image (AVX2) + Vulkan Graphic (Vulkan Temporal)/TAAU + FG

v3.0.0 ships one product library per platform; the v2.1.0 Enable archive and mc_enable.h remain removed. All integrations link libmagic_sr and use mc_nscaler_*. Windows: lib/windows/libmagic_sr.dll, libmagic_sr.lib, libmagic_sr.dll.lib (graphic + Vulkan FG). Linux: lib/linux/libmagic_sr.a and libmagic_sr.so (CPU image + Vulkan Graphic (Vulkan Temporal)/TAAU + FG); rebuild with tools/build_linux_lib.sh and link -lm -lpthread. Do not also link magic_sr_win_vk / magic_vulkan_temporal / magic_vulkan_taau.

2. Image integration (nscaler)

1. Include mc_interface.h and link libmagic_sr.
2. Set ctrl_param_t.model_path and create with mc_nscaler_control(MC_NSCALER_CMD_SET_PARAM), or pass a NULL handle to mc_nscaler_enable for a default 2x SR_SPEED_MODE_FOR_IMAGE create.
3. Bind per-frame width/height on mc_nscaler_input_frame_t; the first enable sets the handle size.
4. Each frame: mc_nscaler_input_frame_t *in2[2] = { &in_frame, NULL }; mc_nscaler_enable(&handle, in2, &out_frame); release only with mc_nscaler_disable.

3. Graphic session integration

5. Include mc_interface.h and link libmagic_sr.

6. Fill ctrl_param_t (graphic alg_mode, scaler in (1, 8], gpu_context, create-stage depth/HDR). ctrl_param_t has no struct_size/width/height; the size comes from the first enable frame.
7. Create with mc_nscaler_control(MC_NSCALER_CMD_SET_PARAM). Vulkan/D3D11 device import is required as documented in mc_interface.h.
8. Create once and keep the handle across frames. Each frame: zero-init temporal_frame_t, set struct_size, bind depth/motion (and optional masks), set jitter/frame_index/camera, attach to mc_nscaler_input_frame_t.frame, bind color/output on mc_nscaler_input_frame_t / mc_nscaler_output_frame_t (including Vulkan layouts on color/depth/motion/output), set command_buffer on Vulkan.
9. mc_nscaler_enable only records or dispatches for that frame. On Vulkan it does not submit or wait. Set reset_history on load/teleport.
10. After the caller has submitted and waited on recorded GPU work, mc_nscaler_disable in shutdown. Do not disable immediately after mc_nscaler_enable while GPU work may still be in flight.

Graphic Super-Resolution

Graphic super-resolution runs through the nscaler session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable. v2.2.0 replaced the v2.0.0 MC_Init / MC_Process / MC_Uninit entry and removed the v2.1.0 MC_Enable helper (mc_enable.h is deleted). A NULL *handle on mc_nscaler_enable creates a session with defaults (2x SR_SPEED_MODE_FOR_IMAGE) and processes the first frame. Pass mc_nscaler_input_frame_t *in2[2] = { &in, NULL } (set in2[1] only for GEN interpolate).

Modes

Value	Enumerator	Role
0	SR_SPEED_MODE_FOR_IMAGE	Image, throughput first
1	SR_BALANCED_MODE_FOR_IMAGE	Image, quality/cost trade-off
2	SR_SPEED_MODE_FOR_GRAPHIC	Graphic, throughput first
3	SR_BALANCED_MODE_FOR_GRAPHIC	Graphic, canonical FSR-family path
4	GEN_SPEED_MODE_FOR_GRAPHIC	FG-Fast (analytic, no weights). Vulkan + MAGIC_SR_HAS_FG. Combinable with SR bits; bare 4 equals SR_SPEED_MODE_FOR_IMAGE|GEN_SPEED.
8	GEN_BALANCED_MODE_FOR_GRAPHIC	FG-CoreV3. Same availability as Fast. Needs magic_fg_core.bin. Do not set both GEN bits.

- Graphic scaler_factor is (1, 8]; 1.0 itself is rejected. Image remains [1.0, 8.0].
- SR_SPEED_MODE_FOR_GRAPHIC on Metal, Vulkan, and OpenGLES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family graphic path (same family as Balanced, not a separate customer API).
- SR_BALANCED_MODE_FOR_GRAPHIC uses the canonical FSR-family path on every GPU graphic backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11,

desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLES do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.

- CPU backends (X86 / NEON) cannot run graphic modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- mc_nscaler_input_frame_t / mc_nscaler_output_frame_t: caller-owned color and output GPU resources.
- mc_nscaler_input_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at create (mc_nscaler_control SET_PARAM, or the first mc_nscaler_enable): Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan graphic requires mc_nscaler_input_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on ctrl_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both graphic modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

3.1 Frame generation integration (Vulkan GEN)

11. Create with SET_PARAM, alg_mode including GEN_SPEED_MODE_FOR_GRAPHIC or GEN_BALANCED_MODE_FOR_GRAPHIC, backend MAGIC_BACKEND_VULKAN, and gpu_context.physical_device + device.

12. For GEN_BALANCED, set ctrl_param_t.model_path to the FG weight file, or leave it empty and ship model/magic_fg_core.bin next to the process working directory or under model/ (or set MAGIC_FG_WEIGHT_PATH). model_path is immutable; recreate the handle to change weights.
13. SR-only frame on a GEN handle: in2[0]=&in0; in2[1]=NULL. Interpolate: fill in0 (pre) and in1 (post); in2[0]=&in0; in2[1]=&in1. Color at output resolution; depth/MV at render resolution on each temporal_frame_t. in0.width/height = render input size. in0.command_buffer = recording VkCommandBuffer.
14. mc_nscaler_enable records only. Caller submits. Output is VK_IMAGE_LAYOUT_GENERAL. Android Vulkan builds and non-Vulkan backends return -100050 / -102013.

- Vulkan Speed (Lite-TAAU) now lets an explicit strong reactive mask suppress the still-camera accumulation branch. Previously that branch ignored the caller reactive mask when the camera was still or per-pixel MV was below about 0.2 px, so pixels with ~0 MV but changing content (TV / video-wall / LED-screen textures) could keep ghosting for about 64–96 frames after a content cut. Strong reactive (≥ about 0.85, fully effective at ≥ 0.95) now weakens that accumulation. Omitting reactive or passing a weak value (< 0.85) is unchanged. Recommendation: mark screen-like objects whose MV is 0 but whose content changes (for example with CustomDepth stencil) as reactive at strength 1.0. Limit: Vulkan only in this release. The Metal / GLES / D3D11 Speed still-accumulation structure is the same idea but is not modified or verified here.

4. Unity / Unreal

Plugins wrap Enable for image upscaling. They do not provide per-frame depth/motion/jitter. Full graphic belongs on the Native session API. See plugin/unity/USAGE_GUIDE.md and plugin/ue/USAGE_GUIDE.md.

5. Vulkan command-buffer notes

- command_buffer must already be recording. MagicSR does not begin, end, submit, or wait.
- layout on color, depth, motion, and image_out must be the actual layout at entry (not 0 / UNDEFINED).
- Keep the handle across frames. mc_nscaler_enable only records. Caller submit/wait, then mc_nscaler_disable at shutdown.
- physical_device, device, and images must be the same VkDevice.